MINI PROJECT REPORT

ON

“HAIKYUUHEIGHTS – BROKER-FREE REAL ESTATE MARKETPLACE”

By

Member 1: [PRN] – [Full Name]
Member 2: [PRN] – [Full Name]
Member 3: [PRN] – [Full Name]

MCA40130 Mini Project
SY MCA
AY 2026–2027

Under the Guidance Of
Mrs. Sukanya Akolkar

Submitted in partial fulfillment of the requirements for the award of the degree of
Master of Computer Applications

MIT World Peace University, Pune
School of Computer Science and Engineering
Department of Computer Science and Applications

Academic Year 2026–2027

CERTIFICATE

This is to certify that _____, student of SY MCA, Semester III, has completed the Mini Project entitled **"HaikyuuHeights – Broker-Free Real Estate Marketplace"** under the guidance of Mrs. Sukanya Akolkar, in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications prescribed under Dr. Vishwanath Karad MIT World Peace University, Pune.

This project report is a record of work carried out during the academic year 2026–2027. The details of the project, implementation, analysis, and testing should be verified by the project team and guide before final submission.

Internal Guide	Program Head – MCA	Program Director
Mrs. Sukanya Akolkar	Dr. P. S. Metkewar	Dr. Anuradha S. Kanade

Date: ____ / ____ / _____

Place: Pune

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mrs. Sukanya Akolkar, for her guidance, encouragement, and valuable suggestions throughout the development of HaikyuuHeights. Her direction helped us organize the project scope and approach its implementation systematically.

We extend our thanks to the Head of the Program, Dr. P. S. Metkewar, and the Program Director, Dr. Anuradha S. Kanade, for providing the academic environment and facilities required for our project work.

We also thank the faculty members of the Department of Computer Science and Applications, MIT World Peace University, for the knowledge and support that contributed to our learning. We acknowledge the contributions of all project team members and classmates who provided feedback and assistance during the development process.

Name: [Team Member 1] [Team Member 2] [Team Member 3]
PRN: [PRN 1] [PRN 2] [PRN 3]
Place: Pune
Date: ____ / ____ / _____

DECLARATION

I / We hereby declare that this Mini Project Report entitled **“HaikyuuHeights – Broker-Free Real Estate Marketplace”** is a record of project work carried out under the supervision of Mrs. Sukanya Akolkar.

I / We further declare that the report describes the analysis, design, implementation, and evaluation of the project to the best of our knowledge. The report has not been submitted previously for the award of any other degree or diploma at this or any other institution. Any third-party tools, frameworks, and references used in the project are acknowledged in the report.

Signature	Team Member 1	Team Member 2	Team Member 3
Name	[Full Name]	[Full Name]	[Full Name]
PRN	[PRN]	[PRN]	[PRN]
Place	Pune		

Date: ____ / ____ / _____

TABLE OF CONTENTS

Section	Title
1	INTRODUCTION
1.1	Existing System, Proposed System and Need and Objective of System
1.2	Scope of Work
1.3	Operating Environment – Hardware and Software
1.4	Description of Technology Used
2	PROPOSED SYSTEM
2.1	Feasibility Study
2.2	User Requirements
3	ANALYSIS AND DESIGN (If Applicable)
3.1	Object Diagram
3.2	Class Diagram
3.3	Use Case Diagram
3.4	Sequence Diagram
3.5	Activity Diagram
3.6	Module Hierarchy Diagram
3.7	User Interface Design (Screens etc.)
3.8	Table Specifications
4	INPUT AND OUTPUT SCREENS
5	TEST PROCEDURES AND IMPLEMENTATION
6	SAMPLE CODE
7	LIMITATIONS
8	FUTURE SCOPE
9	CONCLUSION
10	REFERENCES

Note: Page numbers may change if names, screenshots, diagrams, or other material are added. Update the contents page after final formatting.

1. INTRODUCTION

HaikyuuHeights is a broker-free real estate marketplace implemented as a full-stack web application. It provides public property discovery and role-specific dashboards for buyers, sellers, and administrators. The frontend is built with React and Vite. The backend uses Node.js and Express to expose REST APIs, while MongoDB accessed through Mongoose stores the application's core records.

The application supports property listings, search and filtering, visit bookings, favourites, buyer–seller conversations and messages, reviews, seller verification, commissions, and administrative moderation. Cloudinary is used for image and document media. Supabase Auth is used as the Google OAuth identity provider; it does not replace MongoDB as the application's main database.

1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM

Existing System

Property discovery and coordination can involve separate websites, phone calls, spreadsheets, messaging applications, and in-person arrangements. When property information and interactions are spread across multiple channels, users may find it difficult to compare listings, track visit requests, retain communication history, and understand the status of a request. These are general problems the project seeks to address; the project does not claim that every existing real-estate service has these limitations.

Proposed System

HaikyuuHeights centralizes property discovery and related interactions in one web application. Buyers can browse and filter listings, view property details, book visits, save favourites, communicate with sellers, and submit reviews. Sellers can publish and manage listings, respond to booking requests, and submit verification documents. Administrators can manage users, moderate listings and reviews, review seller verification requests, and monitor commissions and platform activity.

Need for the System

The system is intended to provide an organized channel for property search and buyer–seller coordination. A centralized application can make property information easier to access, keep booking and review records organized, and provide role-specific tools for managing marketplace activity.

Objectives

- Develop a web-based platform for property discovery and buyer–seller interaction.
- Provide search, filtering, and detailed property listing pages.
- Implement registration, login, JWT-based sessions, and role-based access control.
- Support property visit booking and booking status management.
- Allow buyers to save favourites and create property reviews.
- Provide API-backed conversations and messaging between buyers and sellers.
- Support seller document submission and administrator verification.
- Maintain structured application data in MongoDB using Mongoose.
- Use Cloudinary to manage property images and verification documents.
- Provide administrative controls for users, listings, bookings, reviews, verification, and commissions.

1.2 SCOPE OF WORK

Functional Scope

- **Authentication and profiles:** registration, login, profile updates, password changes, Google OAuth, JWT application sessions, and role-aware routing.

- **Property management:** create, update, delete, and display listings, including property details and image management.
- **Discovery:** public property browsing, search, filters, sorting, pagination, and property detail views as supported by the application.
- **Bookings:** buyer visit requests and seller/admin actions to accept, reject, complete, or otherwise manage booking status.
- **Favourites:** save and remove favourite property references.
- **Communication:** create or retrieve conversations and send/read messages through REST endpoints.
- **Reviews:** property and seller reviews, rating display, and administrative moderation.
- **Seller verification:** upload required documents and allow administrators to approve or reject submissions.
- **Commissions:** maintain commission records and expose seller, buyer, and administrative summaries.
- **Administration:** user management, property status management, bookings, reviews, verification requests, and platform activity views.

Scope Boundaries

The source documentation describes messaging as REST/API-backed; it does not establish WebSocket-based real-time messaging. The documented implementation includes commission tracking but does not establish an integrated online payment gateway. Features should be described according to the code that is present, not as planned capabilities.

1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE

Category	Requirement / Technology	Purpose
Operating system	Windows, Linux, or macOS	Development and execution environment
Processor	Intel Core i5 or equivalent (suggested)	Run browser and development tools
Memory	8 GB RAM or more (suggested)	Support IDE, browser, and Node.js processes
Storage	Available space for source, packages, and build artifacts	Project files and dependencies
Frontend	React 18, Vite, JavaScript	User interface and client build
State and routing	Redux Toolkit, React Router	Application state and navigation
HTTP client	Axios	Frontend-to-backend API requests
Backend	Node.js, Express.js	REST endpoints and business logic
Database	MongoDB Atlas, Mongoose	Persist application records
Authentication	JWT, bcryptjs, Supabase Auth	Application sessions, password hashing, Google identity
Media	Cloudinary	Images and verification document storage
Development	Code editor, terminal, browser, npm	Build, run, and debug the application

The frontend runs in a web browser and communicates with the Express API over HTTP. The server loads environment variables, validates required infrastructure settings, connects to MongoDB, and then begins listening. Cloudinary and Supabase require correctly configured credentials and network access.

1.4 DESCRIPTION OF TECHNOLOGY USED

React and Vite

React provides reusable interface components and page composition. Vite provides the frontend development server and production build tooling.

Node.js and Express.js

Node.js runs server-side JavaScript. Express organizes routes and middleware for request parsing, security, authentication, validation, controller execution, and error handling.

MongoDB and Mongoose

MongoDB stores application records as documents. Mongoose provides schemas, models, validation, and database operations for those records.

Redux Toolkit and React Router

Redux Toolkit manages client-side application state and asynchronous API operations. React Router maps URLs to public pages and role-specific dashboards, with protected route components.

Axios and REST APIs

Axios sends HTTP requests from the client to the server. REST endpoints provide operations for authentication, properties, bookings, favourites, conversations, reviews, verification, users, and commissions.

JWT and bcryptjs

The backend signs JWTs for application sessions. Passwords are hashed with bcryptjs before storage and compared during login.

Supabase Auth

Supabase Auth supports Google OAuth identity. The backend validates the Supabase access token, finds or creates the corresponding MongoDB user, and then issues the application's JWT.

Cloudinary

Cloudinary stores property images, profile images, and seller verification media. Local environment secrets and credentials should not be copied into the report.

2. PROPOSED SYSTEM

The proposed system is a client–server marketplace that combines property discovery with role-specific workflows. The client provides the user interface and sends API requests. The server validates requests, checks authentication and authorization requirements, performs business operations, and reads or updates the database. External services provide cloud media storage and Google identity validation.

2.1 FEASIBILITY STUDY

Technical Feasibility

The project uses established web technologies: React, Vite, Node.js, Express, MongoDB, Mongoose, Redux Toolkit, Axios, Supabase Auth, and Cloudinary. The documented source structure separates frontend pages, components, state features, services, routes, backend controllers, models, middleware, and services. This separation supports modular development and maintenance. Technical operation depends on correct environment variables and connectivity to MongoDB, Cloudinary, and Supabase where those services are used.

Economic Feasibility

The project can be developed using open-source frameworks and libraries. Cloud database, identity, hosting, and media services may offer free tiers or paid plans depending on usage and provider policies. Actual project cost depends on the selected plans, storage volume, network usage, hosting configuration, and traffic. No specific monetary saving or production operating cost is claimed in this report.

Operational Feasibility

The application offers separate interfaces and workflows for buyers, sellers, and administrators. Buyers use property search, booking, favourites, communication, and review features. Sellers manage listings, booking requests, and verification submissions. Administrators manage platform users and moderation workflows. This role separation supports organized operation, provided users receive appropriate guidance and the application is deployed with valid configuration.

Schedule Feasibility

The application is organized into independent modules that can be implemented and reviewed in stages, including authentication, property management, booking, favourites, communication, reviews, seller verification, commissions, and administration. A precise schedule and milestone completion dates should be added by the project team if required by the mentor.

Legal and Security Considerations

The system handles user profile data and uploaded media. Deployment should use secure configuration, restrict credentials, apply least-privilege access, validate incoming requests, configure CORS to trusted origins, and protect uploaded documents. Local environment files and secret values must not be included in the report or repository. Applicable privacy and data-protection requirements should be reviewed before public deployment.

2.2 USER REQUIREMENTS

Buyer Requirements

- Register or sign in and manage profile information.
- Browse and search property listings and view property details.
- Request a visit to a property and view booking status.
- Save and remove favourite properties.
- Communicate with sellers through conversations and messages.
- Submit and manage eligible reviews, subject to application rules.

Seller Requirements

- Register or sign in as a seller.

- Create, edit, and delete property listings.
- Upload property images and manage primary images.
- View and manage visit booking requests.
- Submit seller verification documents and view verification status.
- Communicate with buyers and review commission summaries.

Administrator Requirements

- Access the administrative dashboard and platform statistics.
- View and manage users and user status.
- Review property listings and change listing status.
- Review, approve, or reject seller verification submissions.
- Manage bookings and moderate reviews.
- View commission information and recent platform activity.

System Requirements

- Valid environment configuration and MongoDB connectivity.
- Backend API availability and appropriate CORS settings for the frontend origin.
- Correct Cloudinary configuration for media upload and access.
- Supabase configuration and redirect settings for Google OAuth.
- JWT secret configuration and secure handling of credentials.
- Compatible modern web browser and network access to the deployed services.

3. ANALYSIS AND DESIGN (If Applicable)

The diagrams in this section are conceptual views of the documented application architecture and domain. They are intended to communicate major actors, components, and relationships. They should be checked against the final running application and source code before submission.

3.1 OBJECT DIAGRAM

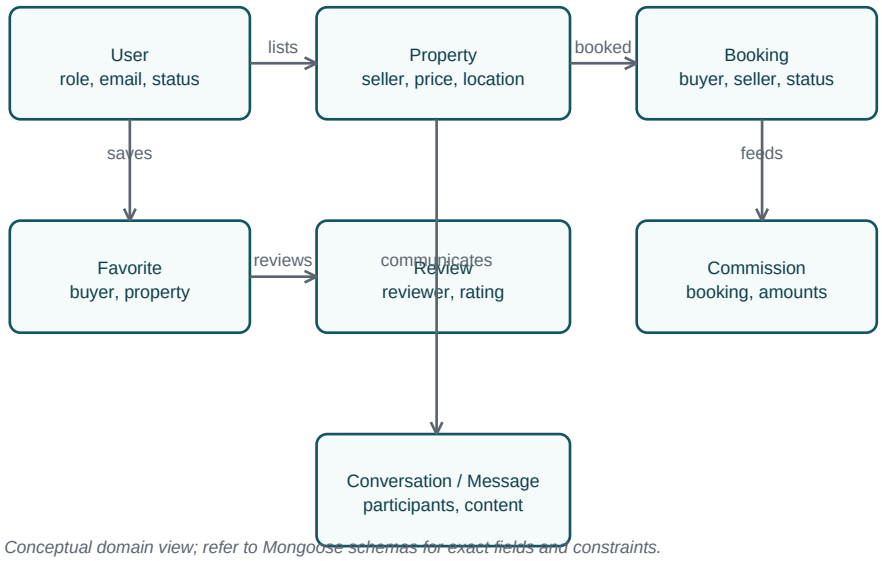
An object diagram represents example instances and their relationships at a particular point in time. The following simplified instance view illustrates a buyer, a property listing, and a visit booking. Values are examples for explanation, not actual production records.

Object / Instance	Illustrative attributes	Relationship
buyer1 : User	role = BUYER; email = buyer@example.com	Creates a booking
property1 : Property	title = Example Apartment; listingType = SALE	Referenced by booking
booking1 : Booking	status = PENDING; requestedDate = example date	Connects buyer, seller, and property
seller1 : User	role = SELLER; verification status varies	Owns the listing and responds to booking

Object relationship summary: buyer1 → booking1 → property1; property1 → seller1. The exact stored fields and status values must be confirmed against the model definitions.

3.2 CLASS DIAGRAM

The following simplified domain diagram highlights principal application models and their conceptual associations. The full schemas contain additional fields and validation rules.



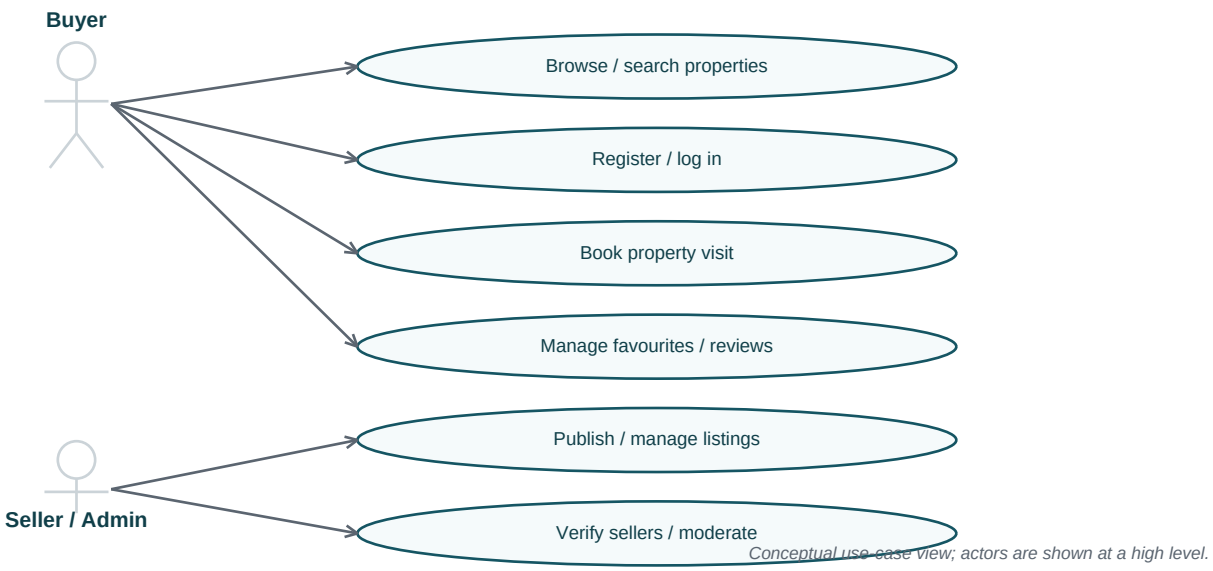
Conceptual domain view; refer to Mongoose schemas for exact fields and constraints.

Model	Principal documented fields / purpose
User	fullName, email, phone, password, role, avatar, isActive, isSuspended
Property	title, description, propertyType, listingType, price, city, state, address, coordinates, room details, media references
Booking	buyer, seller, property, requestedDate, requestedTime, message, status, sellerNote
Favorite	buyer, property
Conversation	property, buyer, seller, lastMessage, lastMessageAt
Message	conversation, sender, content, isRead

Model	Principal documented fields / purpose
Review	reviewer, property, seller, rating, comment
SellerVerification	seller, uploaded document references, status, admin review details
Commission	property, seller, buyer, booking, transaction and commission amounts
Contact	fullName, email, phone, subject, message, status

3.3 USE CASE DIAGRAM

The use-case view summarizes key actions available to the three primary actors. Actual access is governed by route protection, role authorization, and additional seller-verification checks where configured.



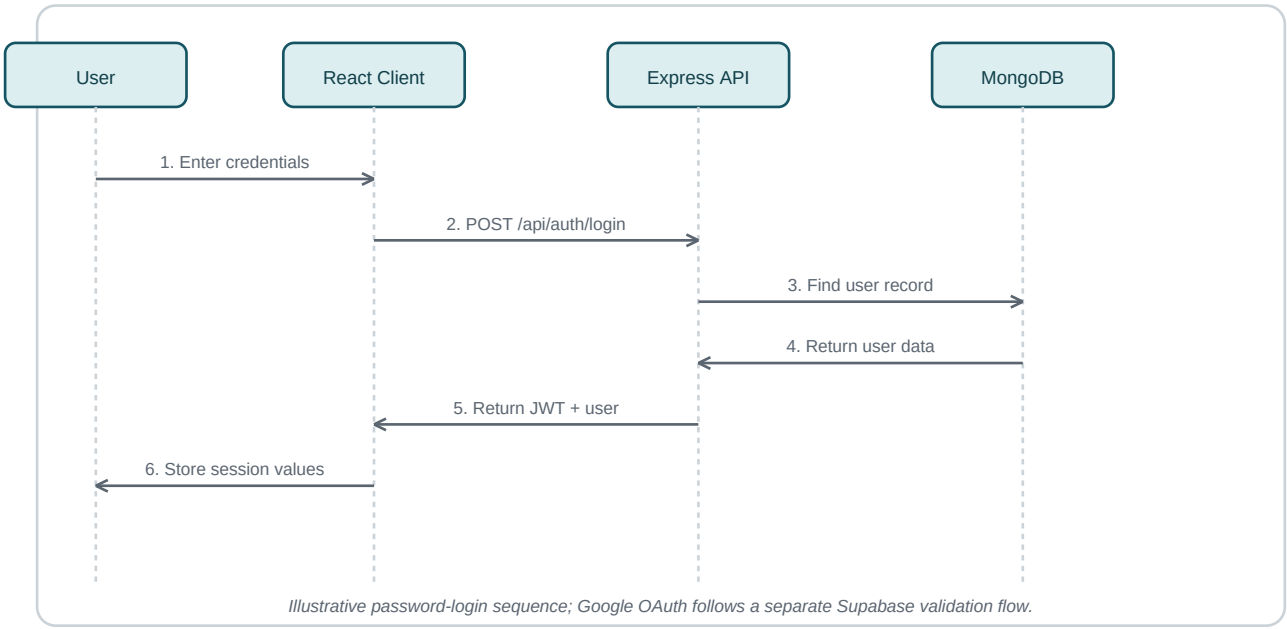
Conceptual use-case view; actors are shown at a high level.

Primary Use Cases

Actor	Main use cases
Buyer	Browse/search properties, view details, manage favourites, request visits, message sellers, submit reviews, manage profile
Seller	Manage property listings and media, manage booking requests, message buyers, submit verification documents, view commission summaries
Administrator	Manage users, moderate property status and reviews, process seller verification, oversee bookings and commissions, view dashboard activity

3.4 SEQUENCE DIAGRAM

The sequence diagram illustrates a simplified password-login request. The backend checks the user record and password, applies account restrictions, and returns application session data after successful authentication. Exact response fields and error messages are determined by the current implementation.



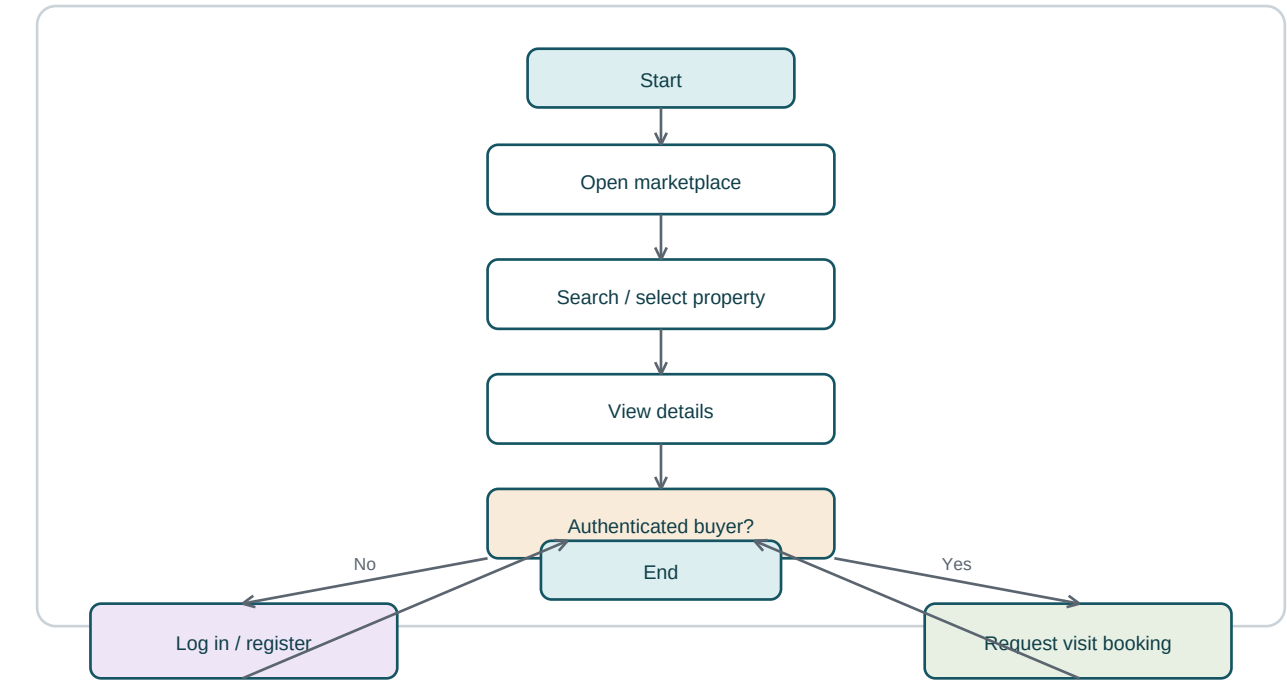
Illustrative password-login sequence; Google OAuth follows a separate Supabase validation flow.

Google OAuth Sequence

1. The browser initiates Google sign-in through the Supabase Auth client.
2. After the redirect, the frontend retrieves the Supabase session and access token.
3. The frontend sends the token to POST /api/auth/supabase.
4. The backend validates the token with Supabase Auth.
5. The backend finds or creates the corresponding MongoDB User record and checks account status.
6. The backend issues an application JWT and returns application user data.
7. The frontend stores the application session values and ends the temporary Supabase browser session.

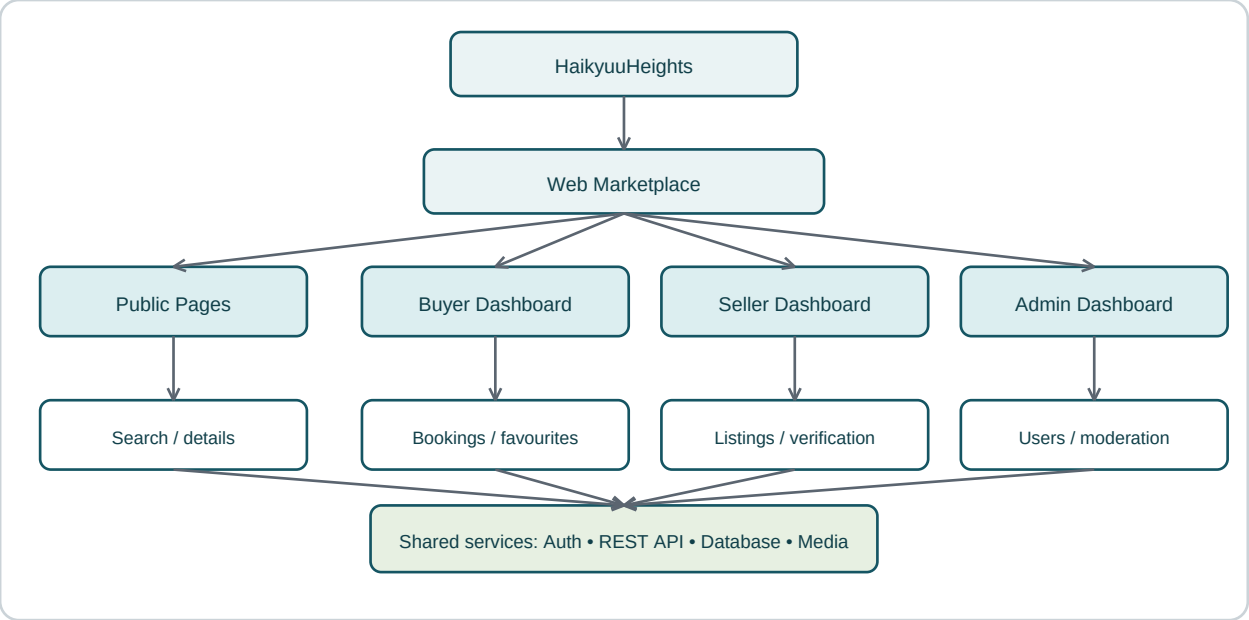
3.5 ACTIVITY DIAGRAM

The activity diagram illustrates a typical buyer journey from property discovery to a visit request. It is a high-level workflow and does not represent every possible validation or error branch.



3.6 MODULE HIERARCHY DIAGRAM

The module hierarchy groups public pages and role-specific dashboards under the marketplace application. Shared authentication, API services, database models, middleware, and media integration support the user-facing modules.



3.7 USER INTERFACE DESIGN (SCREENS ETC.)

The project documentation identifies public pages and separate dashboards for buyers, sellers, and administrators. The source inventory includes the following screens and reusable interface components. Actual screenshots from the running application should be inserted in this section before submission; the architecture PDF supplied for this report did not contain usable application screenshots.

Screen / Interface	Purpose	Suggested screenshot
Home page	Entry point and featured property discovery	Insert Figure 3.7.1: Home page
Properties page	Browse listings, search, sort, and apply filters	Insert Figure 3.7.2: Property search
Property details	View listing details, images, and related actions	Insert Figure 3.7.3: Property details
Login / Registration	Authenticate users and register accounts	Insert Figure 3.7.4: Authentication
Buyer dashboard	Overview of buyer activity, bookings, favourites, reviews	Insert Figure 3.7.5: Buyer dashboard
Seller dashboard	Manage listings, bookings, profile, and verification	Insert Figure 3.7.6: Seller dashboard
Add/Edit property	Submit or modify property data and images	Insert Figure 3.7.7: Property form
Messaging	View conversations and send messages	Insert Figure 3.7.8: Messaging
Admin dashboard	Monitor platform activity and manage modules	Insert Figure 3.7.9: Admin dashboard
Verification management	Review seller document submissions	Insert Figure 3.7.10: Verification screen

3.8 TABLE SPECIFICATIONS

MongoDB is the application's primary data store. The following table lists principal Mongoose model entities and their documented top-level fields or purpose. MongoDB stores documents rather than relational SQL tables; the term “table specifications” is used here to satisfy the report format.

Collection / Model	Principal fields or data purpose
User	fullName, email, phone, password hash, role, avatar, isActive, isSuspended
Property	title, description, propertyType, listingType, price, location, bedrooms, bathrooms, image references, seller reference
Booking	buyer, seller, property, requestedDate, requestedTime, message, status, sellerNote
Commission	property, seller, buyer, booking, transactionAmount, commission rates and amounts
Contact	fullName, email, phone, subject, message, status
Conversation	property, buyer, seller, lastMessage, lastMessageAt
Favorite	buyer, property
Message	conversation, sender, content, isRead
Review	reviewer, property, seller, rating, comment
SellerVerification	seller, document/media references, file metadata, status, adminNote, reviewedBy, reviewedAt

Relationships are represented through document references and model associations. A user may own property listings, create bookings, save favourites, and participate in reviews or conversations. A booking connects a buyer, seller, and property. Conversations contain messages, and eligible completed transactions can be associated with commission records.

4. INPUT AND OUTPUT SCREENS

This section describes the major input forms and output screens identified in the application source documentation. Replace the figure placeholders with screenshots captured from the actual running application. Do not use mock screenshots as evidence of implemented functionality.

4.1 Input Screens

Registration and Login

The authentication interface accepts user details for registration or login. Registration includes identity and credential fields documented in the application, including full name, email, phone, password, confirmation password, and an optional role. Login normalizes the email and compares the supplied password with the stored hash. Suspended users are rejected.

Property Listing Form

The seller property form collects property information such as title, description, property type, listing type, price, location, property features, and image uploads. The backend validates incoming data and uses upload middleware and Cloudinary for supported media operations.

Booking Request Form

The buyer booking workflow submits a property reference and requested visit information. The application tracks booking status and allows authorized users to perform supported actions such as accepting, rejecting, completing, or cancelling a booking according to controller rules.

Review Form

The review interface collects a rating and comment associated with a property and seller. Server-side rules validate the request, and review management endpoints support the documented update and deletion operations.

Seller Verification Form

The seller verification interface collects required identity or ownership documents and submits them for administrative review. The system stores media references and verification status information.

4.2 Output Screens

- **Property listing output:** displays property cards, listing information, search/filter results, and pagination where available.
- **Property details output:** presents detailed listing information and associated images.
- **Buyer dashboard output:** summarizes buyer activities such as bookings and reviews.
- **Seller dashboard output:** presents seller listings, booking requests, verification status, and commission summaries.
- **Administrator output:** provides dashboard statistics, recent activity, property moderation, user management, verification management, bookings, reviews, and commissions.
- **Conversation output:** displays conversation threads and messages retrieved through REST endpoints.
- **Profile output:** shows profile information and supported account management actions.

4.3 Screenshot Placement Checklist

Figure	Required evidence	Status
4.1	Home page and property discovery	Insert actual screenshot
4.2	Property listing/search results	Insert actual screenshot
4.3	Property detail page	Insert actual screenshot
4.4	Login or registration screen	Insert actual screenshot
4.5	Buyer dashboard / bookings	Insert actual screenshot
4.6	Seller listing management	Insert actual screenshot
4.7	Seller verification screen	Insert actual screenshot
4.8	Administrator dashboard	Insert actual screenshot

Screenshots are not embedded in this generated report because the supplied project document describes source architecture rather than providing screenshots of the live UI. Capture them from your running application and insert them with figure numbers and captions.

5. TEST PROCEDURES AND IMPLEMENTATION

The implementation uses a React/Vite frontend and an Express API backend. The backend request pipeline includes security headers, CORS, request parsing, rate limiting, route-level authentication where configured, role authorization, validation, controller execution, and centralized error handling. Controllers use consistent success and error response helpers, while asynchronous errors are forwarded to centralized error middleware.

5.1 Implementation Overview

Layer	Implementation responsibilities
Frontend bootstrap	Mount React, Redux Provider, router, global CSS, and notifications
Routing and access	Map public and dashboard URLs; protect routes and enforce role-based access
State management	Maintain loading/error state and domain data using Redux Toolkit features
API services	Use Axios and service modules to call backend endpoints and attach JWTs
Backend routing	Declare REST endpoints and apply relevant middleware
Controllers	Coordinate business operations for users, properties, bookings, reviews, and other domains
Models	Define Mongoose schemas and persistence rules
External services	Use Cloudinary for media and Supabase Auth for Google identity validation
Error handling	Map expected application, validation, database, and JWT errors to HTTP responses

5.2 Test Procedures

The supplied architecture documentation states that no automated test suite or test script was present in the analyzed source inventory. Therefore, the following is a recommended manual verification plan, not a claim that every test has already passed. Record the actual result after executing each test in the running application.

Test ID	Test procedure	Expected result	Actual result
TC-01	Register with valid required fields	Account is created and appropriate response is returned	To be executed
TC-02	Attempt login with correct credentials	Application returns authenticated user data and JWT	To be executed
TC-03	Attempt login with incorrect password	Request is rejected without issuing a valid session	To be executed
TC-04	Access protected endpoint without JWT	Request is rejected according to authorization rules	To be executed
TC-05	Access role-restricted page with wrong role	Access is denied or redirected appropriately	To be executed
TC-06	Create a property with valid data	Listing is saved and returned to authorized seller	To be executed
TC-07	Submit invalid property data or media	Validation rejects invalid input	To be executed
TC-08	Search/filter properties	Results reflect supported filters and query parameters	To be executed
TC-09	Create and cancel a booking	Booking state changes according to access rules	To be executed
TC-10	Add/remove a favourite	Saved property collection updates	To be executed
TC-11	Submit review with valid and invalid data	Server-side review rules are enforced	To be executed
TC-12	Submit seller verification request	Document references and status are recorded	To be executed

Test ID	Test procedure	Expected result	Actual result
TC-13	Admin approves/rejects verification	Verification status changes for authorized admin	To be executed
TC-14	Upload property image	Supported image is uploaded and associated with listing	To be executed
TC-15	Google OAuth sign-in	Supabase token is validated and application JWT is issued	To be executed
TC-16	Database unavailable at startup	Server does not start successfully without required MongoDB connection	To be executed

5.3 Security and Reliability Considerations

- Passwords are hashed using bcryptjs and are not included in normal user responses.
- JWTs are signed by the backend and sent in the Authorization header for protected requests.
- Role middleware restricts buyer, seller, and administrator operations.
- Rate limiting is configured globally and more strictly for authentication routes where configured.
- Request validation is used on relevant authentication and domain request paths.
- Centralized error middleware handles application and selected database/authentication errors.
- Environment variables are used for infrastructure credentials; secrets must not be committed or included in reports.
- Production deployment should use exact CORS origins, secure HTTPS redirect configuration, and a review of upload security.

5.4 Test Result Reporting

After executing the test cases, replace each “To be executed” entry with Pass, Fail, or Not Applicable and record any issue observed. Do not claim automated testing or successful execution unless supported by actual test evidence. If manual testing is completed, identify the browser, environment, test date, and the version of the application used.

6. SAMPLE CODE

The examples below illustrate the documented architecture and common API patterns. They are explanatory samples rather than a verbatim export of the repository. Before final submission, compare them with the actual source code and replace them with exact implementation excerpts if the mentor requires code copied directly from the project.

6.1 Express Application Route Mounting

```
/* Illustrative excerpt: Express API route mounting */
import express from "express";
import authRoutes from "./routes/auth.routes.js";
import propertyRoutes from "./routes/property.routes.js";
import bookingRoutes from "./routes/booking.routes.js";

const app = express();

app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/property", propertyRoutes);
app.use("/api/booking", bookingRoutes);

export default app;
```

Purpose: routes group related API endpoints and delegate request handling to route middleware and controllers. The actual application also configures security headers, CORS, rate limiting, health checks, error handling, and additional route modules.

6.2 JWT Authorization Header in Client Requests

```
/* Illustrative Axios request pattern */
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("hh_token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export default api;
```

Purpose: the client includes the application's JWT in the Authorization header for protected API requests. The source documentation identifies hh_token as the client-side token key; verify the precise interceptor implementation in the repository.

6.3 Mongoose Model Concept

```
/* Simplified illustrative Mongoose model */
import mongoose from "mongoose";

const propertySchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String },
  propertyType: { type: String },
  listingType: { type: String },
  price: { type: Number },
  city: { type: String },
  state: { type: String }
}, { timestamps: true });

export default mongoose.model("Property", propertySchema);
```

Purpose: Mongoose schemas describe document fields and validation. This is a simplified illustration; the actual Property schema includes additional fields, image metadata, seller references, and validation rules.

6.4 Authentication Workflow (Pseudocode)

```
1. Receive email and password.
2. Normalize the email and locate the user record.
3. Retrieve the stored password hash.
4. Compare the submitted password with bcrypt.
5. Reject invalid credentials or suspended accounts.
6. Sign an application JWT containing user identifier and role.
7. Return the token and appropriate user data.
```

Purpose: this summarizes the documented password-login flow. The exact implementation and response format should be taken from the current authentication controller.

6.5 Property API Examples

Method	Endpoint	Purpose
GET	/api/property/	Retrieve property listings
GET	/api/property/:id	Retrieve a specific property
POST	/api/property/	Create a property listing
PATCH	/api/property/:id	Update a property listing
DELETE	/api/property/:id	Delete a property listing
GET	/api/property/seller/mine	Retrieve the authenticated seller's listings

6.6 Authentication API Examples

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user
POST	/api/auth/login	Authenticate using credentials
POST	/api/auth/supabase	Exchange validated Supabase identity for application session
GET	/api/auth/me	Retrieve current authenticated user
PATCH	/api/auth/profile	Update profile information
PATCH	/api/auth/change-password	Change account password

Do not include live secrets, private API keys, passwords, database connection strings, or production JWT secrets in this section.

7. LIMITATIONS

The following limitations are based on the supplied code-derived architecture review and the boundaries of the documented implementation.

1. The application requires MongoDB connectivity at startup. Supabase Auth is used for Google identity validation and does not replace MongoDB as the application database.

2. Google OAuth depends on correct Supabase provider configuration, backend availability, MongoDB connectivity, redirect URLs, and CORS configuration.

3. The documented messaging implementation uses REST APIs. WebSocket-based real-time delivery is not established by the reviewed source inventory.

4. The reviewed source inventory did not contain an automated test suite or test script. Validation should be reported as manual/development verification unless automated tests are added and executed.

5. JWTs are stored in browser localStorage in the documented implementation. This should be assessed as a security trade-off for production deployment; secure httpOnly cookies and refresh-token rotation may be considered.

6. Input validation should be reviewed across every mutating route to ensure consistent server-side enforcement and schema-level constraints.

7. Cloud media upload requires correct configuration and should be strengthened with content inspection, quotas, and additional production controls where appropriate.

8. Online payment gateway processing is not established by the supplied architecture documentation. Commission tracking should not be described as payment processing.

9. Actual performance, availability, scalability, and cost depend on deployment configuration, service plans, network conditions, and usage.

10. The supplied architecture document did not include screenshots of the running user interface; real application screenshots must be added to the report.

8. FUTURE SCOPE

The following enhancements could be considered in future iterations. They are recommendations and should not be presented as features already implemented.

1. Add secure session improvements such as httpOnly cookies, refresh-token rotation, session revocation, and stronger token lifecycle management.

2. Introduce automated unit tests, API integration tests, and end-to-end tests for authentication, role access, listing management, booking workflows, and verification.

3. Add WebSocket-based or other event-driven messaging if real-time communication is required.

4. Integrate a payment gateway with verified transaction callbacks, payment reconciliation, refunds, and appropriate compliance controls if online payments become a requirement.

5. Enhance search with geospatial filtering, map-based discovery, saved searches, and personalized property recommendations.

6. Add notifications for booking updates, seller verification decisions, messages, and review activity.

7. Improve media security with upload quotas, stronger file-content validation, malware scanning, and controlled access to private verification documents.

8. Introduce structured audit trails for administrative actions and sensitive account changes.

9. Improve observability with centralized logs, metrics, alerts, health checks, and error monitoring.

10. Optimize performance through caching, database indexes, pagination review, query profiling, and CDN configuration where appropriate.

11. Add deployment automation, CI/CD workflows, environment-specific configuration, backups, and disaster-recovery procedures.

12. Conduct usability and accessibility reviews and add localization support if required.

9. CONCLUSION

HaikyuuHeights is a full-stack, broker-free real estate marketplace designed to organize property discovery and buyer–seller interactions in a single web application. The project combines a React/Vite frontend with a Node.js and Express backend, MongoDB and Mongoose for application data, Cloudinary for media storage, and Supabase Auth for Google identity authentication.

The documented system provides core marketplace workflows including property listing and discovery, visit bookings, favourites, API-backed conversations and messages, reviews, seller verification, commission tracking, and administrative moderation. Its role-specific interfaces and backend authorization mechanisms help separate buyer, seller, and administrator responsibilities.

The project demonstrates the integration of frontend state management, REST API design, document-oriented persistence, authentication, authorization, cloud media handling, and modular application architecture. Further improvements can focus on automated testing, production security, observability, scalability, and additional user-facing capabilities.

Before final academic submission, the team should verify all report details against the running application, add actual screenshots, replace member placeholders, execute and document the test cases, and confirm that all diagrams accurately reflect the final codebase.

10. REFERENCES

1. HaikyuuHeights – Complete Software Architecture and Codebase Documentation, mentor review handoff document, generated 27 October 2026.

2. Mini Project: Project Report Submission Guidelines, SY MCA, MIT World Peace University, academic year 2026–2027.

3. React Documentation. https://react.dev/

4. Vite Documentation. https://vite.dev/

5. Node.js Documentation. https://nodejs.org/docs/

6. Express Documentation. https://expressjs.com/

7. MongoDB Documentation. https://www.mongodb.com/docs/

8. Mongoose Documentation. https://mongoosejs.com/docs/

9. Redux Toolkit Documentation. https://redux-toolkit.js.org/

10. React Router Documentation. https://reactrouter.com/

11. Axios Documentation. https://axios-http.com/

12. Supabase Documentation – Authentication. https://supabase.com/docs/guides/auth

13. Cloudinary Documentation. https://cloudinary.com/documentation

14. JSON Web Tokens – Introduction. https://jwt.io/introduction

15. bcrypt.js project documentation. https://github.com/dcodeIO/bcrypt.js

End of Report